

Aegis AML — API Specification

Version: 1.0 — MVP **Base URL:** <https://api.aegis-aml.com> (local: <http://localhost:8000>) **Last Updated:** 2026-06-10

Status — original MVP design (2026-06-10). Several endpoints changed during implementation. For the authoritative, code-verified API reference, see [AEGIS_KNOWLEDGE_BASE.md](#) §13 (paths) and Appendix A (real request/response shapes); the deviations are catalogued in §21. Where this document and the code disagree, the code wins. Notable changes already in the code:

- Ingestion is `POST /api/v1/ingest/` returning `200` — not `POST /api/v1/alerts/ingest` → `202`.
- The review queue lives under `/api/v1/alerts/queue...` (not `/api/v1/queue...`).
- The simulator is `POST /api/v1/alerts/simulator/submit-test-alert`.
- There is no standalone `/api/v1/webhooks/sink/...` router; sink events are read via `GET /api/v1/tenant/webhook/events`.
- Live handlers return FastAPI's `{detail: ...}` error shape (same HTTP codes), not the `{error: {code,message}}` envelope shown below.

Table of contents

- [Auth Schemes](#)
- [Standard Error Response](#)
- [Domain 1: Authentication](#)
 - [POST /api/v1/auth/signup](#)
 - [POST /api/v1/auth/login](#)
 - [POST /api/v1/auth/refresh](#)
 - [GET /api/v1/auth/me](#)
- [Domain 2: Super Admin](#)
 - [GET /api/v1/admin/verifications](#)
 - [POST /api/v1/admin/tenants/{tenant_id}/approve](#)
 - [POST /api/v1/admin/tenants/{tenant_id}/reject](#)
 - [GET /api/v1/admin/tenants](#)
 - [POST /api/v1/admin/tenants/{tenant_id}/suspend](#)
 - [POST /api/v1/admin/tenants/{tenant_id}/reinstate](#)
 - [GET /api/v1/admin/logs](#)
 - [GET /api/v1/admin/groq-usage](#)
- [Domain 3: Tenant Configuration](#)
 - [GET /api/v1/tenant/profile](#)
 - [GET /api/v1/tenant/credentials](#)
 - [POST /api/v1/tenant/credentials/rotate](#)
 - [GET /api/v1/tenant/webhook](#)
 - [PUT /api/v1/tenant/webhook](#)
 - [POST /api/v1/tenant/webhook/test](#)
 - [GET /api/v1/tenant/schemas](#)

- [POST /api/v1/tenant/schemas/select-preset](#)
 - [GET /api/v1/tenant/llm-config](#)
 - [PUT /api/v1/tenant/llm-config](#)
 - [GET /api/v1/tenant/usage](#)
 - [Domain 4: Alert Ingestion](#)
 - [POST /api/v1/alerts/ingest](#)
 - [Domain 5: SAR Review Queue](#)
 - [GET /api/v1/queue](#)
 - [GET /api/v1/queue/{alert_id}](#)
 - [PUT /api/v1/queue/{alert_id}/draft](#)
 - [GET /api/v1/queue/{alert_id}/preview-rehydrated](#)
 - [POST /api/v1/queue/{alert_id}/approve](#)
 - [POST /api/v1/queue/{alert_id}/reject](#)
 - [Domain 6: Webhook Sink \(Built-in Test Receiver\)](#)
 - [POST /api/v1/webhooks/sink/{tenant_id_public}](#)
 - [GET /api/v1/webhooks/sink/{tenant_id_public}/events](#)
 - [Domain 7: Simulator](#)
 - [POST /api/v1/simulator/submit-test-alert](#)
 - [Webhook Payload Verification \(Client-Side Reference\)](#)
 - [Rate Limits \(MVP defaults, not enforced per-tenant\)](#)
-

Auth Schemes

Two authentication mechanisms are used:

1. Bearer JWT (Portal users) All portal endpoints (client portal + admin dashboard) use JWT.

```
Authorization: Bearer <access_token>
```

2. API Key (Headless ingestion) The alert ingestion endpoint uses API key + tenant ID headers:

```
X-API-Key: sk-ae-a1b2c3d4e5f6...
X-Tenant-ID: TEN-0001
```

Standard Error Response

All errors follow this shape:

```
{
  "error": {
    "code": "INVALID_API_KEY",
    "message": "The provided API key is invalid or has been revoked.",
    "details": {}
  }
}
```

```
}  
}
```

Common error codes:

Code	HTTP	Meaning
UNAUTHORIZED	401	Missing or invalid credentials
FORBIDDEN	403	Authenticated but insufficient role
NOT_FOUND	404	Resource does not exist
VALIDATION_ERROR	422	Request body failed validation
TENANT_SUSPENDED	403	Tenant's API key is suspended
TENANT_NOT_ACTIVE	403	Tenant is still pending/rejected
RATE_LIMITED	429	Too many requests
INTERNAL_ERROR	500	Server-side failure

Domain 1: Authentication

POST /api/v1/auth/signup

Register a new tenant + admin user. Sets status to **PENDING_VERIFICATION**.

Auth: None

Request:

```
{  
  "company_name": "PayFast India Pvt Ltd",  
  "company_type": "FINTECH",  
  "cin": "U74999MH2021PTC123456",  
  "sebi_reg_no": null,  
  "website": "https://payfast.in",  
  "admin_name": "Nikhil Karur",  
  "admin_email": "nikhil@payfast.in",  
  "admin_phone": "+919876543210",  
  "admin_designation": "Head of Compliance",  
  "password": "SecurePass123!"  
}
```

Response 201:

```
{  
  "tenant_id_public": null,  
  "status": "PENDING_VERIFICATION",  
}
```

```
"message": "Application submitted successfully. You will be notified once
verified.",
"user": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "nikhil@payfast.in",
  "full_name": "Nikhil Karur",
  "role": "TENANT_ADMIN"
}
```

Business Logic:

1. Validate uniqueness of email
2. Create **tenant** record with status **PENDING_VERIFICATION**
3. Create **user** record with role **TENANT_ADMIN**, linked to tenant
4. Log **TENANT_SIGNUP** to audit_logs
5. Return JWT (access + refresh) so user can immediately see their status page

POST /api/v1/auth/login

Auth: None

Request:

```
{
  "email": "nikhil@payfast.in",
  "password": "SecurePass123!"
}
```

Response 200:

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "bearer",
  "expires_in": 900,
  "user": {
    "id": "550e8400-...",
    "email": "nikhil@payfast.in",
    "full_name": "Nikhil Karur",
    "role": "TENANT_ADMIN",
    "tenant_id": "uuid",
    "tenant_status": "ACTIVE",
    "tenant_name": "PayFast India Pvt Ltd",
    "tenant_id_public": "TEN-0001"
  }
}
```

POST /api/v1/auth/refresh

Auth: None (refresh token in body)

Request:

```
{ "refresh_token": "eyJ..." }
```

Response 200:

```
{  
  "access_token": "eyJ...",  
  "expires_in": 900  
}
```

GET /api/v1/auth/me

Auth: Bearer JWT

Response 200:

```
{  
  "id": "uuid",  
  "email": "nikhil@payfast.in",  
  "full_name": "Nikhil Karur",  
  "role": "TENANT_ADMIN",  
  "tenant": {  
    "id": "uuid",  
    "name": "PayFast India Pvt Ltd",  
    "status": "ACTIVE",  
    "tenant_id_public": "TEN-0001",  
    "company_type": "FINTECH"  
  }  
}
```

Domain 2: Super Admin

All endpoints require role **SUPER_ADMIN**.

GET /api/v1/admin/verifications

Pending tenant applications.

Query Params: `?page=1&per_page=20`

Response 200:

```
{
  "items": [
    {
      "id": "uuid",
      "name": "PayFast India Pvt Ltd",
      "company_type": "FINTECH",
      "cin": "U74999MH2021PTC123456",
      "website": "https://payfast.in",
      "admin_name": "Nikhil Karur",
      "admin_email": "nikhil@payfast.in",
      "admin_phone": "+919876543210",
      "created_at": "2026-06-10T09:00:00Z"
    }
  ],
  "total": 3,
  "page": 1,
  "per_page": 20
}
```

POST /api/v1/admin/tenants/{tenant_id}/approve

Auth: SUPER_ADMIN Bearer JWT

Request: Empty body

Response 200:

```
{
  "message": "Tenant approved successfully.",
  "tenant": {
    "id": "uuid",
    "name": "PayFast India Pvt Ltd",
    "status": "ACTIVE",
    "tenant_id_public": "TEN-0001"
  },
  "credentials": {
    "api_key": "sk-ae-a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0",
    "api_key_prefix": "sk-ae-a1b2",
    "tenant_id": "TEN-0001",
    "warning": "This is the only time the full API key will be shown. Store it securely."
  }
}
```

Business Logic:

1. Validate tenant is in **PENDING_VERIFICATION** status
 2. Generate cryptographically secure API key (**sk-ae-** + 34 hex chars)
 3. bcrypt-hash and store the key; store only the prefix for display
 4. Generate **tenant_id_public** (format: **TEN-XXXX**, auto-incremented)
 5. Update tenant status to **ACTIVE**, set **approved_at**, **approved_by**
 6. Create default **webhook_configs** (use_internal_sink=true, is_active=true)
 7. Create default **llm_configs** (provider=GROQ, model=llama-3.3-70b-versatile)
 8. Log **TENANT_APPROVED** + **API_KEY_GENERATED** to audit_logs
 9. Return full plaintext API key (ONCE ONLY)
-

POST /api/v1/admin/tenants/{tenant_id}/reject

Request:

```
{ "reason": "Unable to verify corporate registration number." }
```

Response 200:

```
{ "message": "Tenant rejected.", "status": "REJECTED" }
```

GET /api/v1/admin/tenants

Query Params: ?status=ACTIVE&page=1&per_page=20&search=payfast

Response 200:

```
{
  "items": [
    {
      "id": "uuid",
      "name": "PayFast India Pvt Ltd",
      "tenant_id_public": "TEN-0001",
      "company_type": "FINTECH",
      "status": "ACTIVE",
      "total_alerts": 47,
      "approved_sars": 31,
      "joined_at": "2026-06-01T00:00:00Z",
      "last_active_at": "2026-06-10T08:45:00Z"
    }
  ],
  "total": 1,
  "page": 1,
  "per_page": 20
}
```

POST /api/v1/admin/tenants/{tenant_id}/suspend

Request: Empty body

Response 200:

```
{ "message": "Tenant API key suspended. All ingestion requests will be rejected."
}
```

POST /api/v1/admin/tenants/{tenant_id}/reinstate

Response 200:

```
{ "message": "Tenant reinstated. API key is active." }
```

GET /api/v1/admin/logs

Query Params: ?tenant_id=uuid&endpoint=/api/v1/alerts/ingest&status_code=500&from=2026-06-01&to=2026-06-10&page=1&per_page=50

Response 200:

```
{
  "items": [
    {
      "id": "uuid",
      "tenant_id": "uuid",
      "tenant_name": "PayFast India Pvt Ltd",
      "method": "POST",
      "endpoint": "/api/v1/alerts/ingest",
      "status_code": 200,
      "latency_ms": 342,
      "request_ip": "52.66.12.34",
      "created_at": "2026-06-10T08:45:00Z"
    }
  ],
  "total": 154
}
```

GET /api/v1/admin/groq-usage

Response 200:


```
{
  "total_tokens_all_time": 2847593,
  "total_tokens_this_month": 184720,
  "estimated_cost_usd_this_month": 0.28,
  "per_tenant": [
    {
      "tenant_id": "uuid",
      "tenant_name": "PayFast India Pvt Ltd",
      "tokens_this_month": 120000,
      "total_requests": 89
    }
  ]
}
```

Domain 3: Tenant Configuration

All endpoints require Bearer JWT with role `TENANT_ADMIN`. All operations are scoped to the JWT's `tenant_id`.

GET `/api/v1/tenant/profile`

Response 200:

```
{
  "id": "uuid",
  "name": "PayFast India Pvt Ltd",
  "company_type": "FINTECH",
  "cin": "U74999MH...",
  "website": "https://payfast.in",
  "status": "ACTIVE",
  "tenant_id_public": "TEN-0001",
  "api_key_prefix": "sk-ae-a1b2",
  "joined_at": "2026-06-01T00:00:00Z"
}
```

GET `/api/v1/tenant/credentials`

Returns visible credential info (not the full API key — that's gone).

Response 200:

```
{
  "tenant_id_public": "TEN-0001",
  "api_key_prefix": "sk-ae-a1b2",
  "api_key_last_rotated": null
}
```

POST /api/v1/tenant/credentials/rotate

Generates a new API key, invalidating the old one.

Request:

```
{ "confirm_password": "SecurePass123!" }
```

Response 200:

```
{
  "new_api_key": "sk-ae-z9y8x7w6v5u4t3s2r1q0p9o8n7m6l5k4j3i2h1",
  "api_key_prefix": "sk-ae-z9y8",
  "warning": "Your old API key is now invalid. Update your integration immediately."
}
```

GET /api/v1/tenant/webhook

Response 200:

```
{
  "callback_url": "https://payfast.in/aegis/callback",
  "use_internal_sink": false,
  "internal_sink_url": "https://api.aegis-aml.com/api/v1/webhooks/sink/TEN-0001",
  "secret_prefix": "wh-sec-a1b2",
  "last_tested_at": "2026-06-10T07:00:00Z",
  "last_test_status": "SUCCESS"
}
```

PUT /api/v1/tenant/webhook

Request:

```
{
  "callback_url": "https://payfast.in/aegis/callback",
  "use_internal_sink": false
}
```

Response 200:

```
{
  "message": "Webhook configuration updated.",
  "callback_url": "https://payfast.in/aegis/callback",
  "use_internal_sink": false,
  "new_secret": "wh-sec-z9y8x7w6v5u4t3s2r1q0p9o8n7m6...",
  "secret_prefix": "wh-sec-z9y8",
  "warning": "New webhook secret generated. Update your HMAC verification immediately."
}
```

Note: Every time webhook config is updated, a new secret is generated and returned once.

POST /api/v1/tenant/webhook/test

Sends a mock SAR payload to the configured webhook destination.

Request: Empty body

Response 200:

```
{
  "message": "Test payload sent.",
  "destination": "internal_sink",
  "delivery_status": "DELIVERED",
  "http_status": null,
  "sink_event_id": "uuid"
}
```

GET /api/v1/tenant/schemas

Response 200:

```
{
  "active_schema": {
    "id": "uuid",
    "name": "Standard Fintech Transaction Alert",
    "template_key": "STANDARD_FINTECH"
  },
  "available_presets": [
    {
      "template_key": "STANDARD_FINTECH",
      "name": "Standard Fintech Transaction Alert",
      "description": "Covers UPI/NEFT/RTGS/IMPS transactions from digital lending, wallets, and payment apps."
    },
    {

```

```
    "template_key": "SEBI_BROKER",
    "name": "SEBI Stock Broker Trading Alert",
    "description": "Covers equity, F&O, and commodity trades. Uses PAN and Demat
account identifiers."
  },
  {
    "template_key": "PAYMENT_GW",
    "name": "Payment Gateway Alert",
    "description": "Covers payment gateway transactions with UPI VPA, merchant
ID, and device fingerprint."
  }
]
```

POST /api/v1/tenant/schemas/select-preset

Request:

```
{ "template_key": "SEBI_BROKER" }
```

Response 200:

```
{
  "message": "Schema updated to SEBI Stock Broker Trading Alert.",
  "schema_id": "uuid"
}
```

GET /api/v1/tenant/llm-config

Response 200:

```
{
  "provider": "GROQ",
  "model_name": "llama-3.3-70b-versatile",
  "sar_template_style": "BOTH",
  "total_tokens_used": 120000,
  "total_requests": 89
}
```

PUT /api/v1/tenant/llm-config

Request:

```
{ "sar_template_style": "NARRATIVE" }
```

Response 200:

```
{ "message": "LLM configuration updated.", "sar_template_style": "NARRATIVE" }
```

GET /api/v1/tenant/usage**Response 200:**

```
{
  "period": "last_30_days",
  "alerts_ingested": 47,
  "sars_approved": 31,
  "alerts_rejected": 6,
  "false_positives_cleared": 10,
  "avg_review_time_minutes": 8.3,
  "daily_breakdown": [
    { "date": "2026-06-01", "alerts": 3, "approved": 2 },
    ...
  ]
}
```

Domain 4: Alert Ingestion

POST /api/v1/alerts/ingest

The primary headless API. Called by the fintech's TMS.

Auth: API Key headers

```
X-API-Key: sk-ae-a1b2c3d4...
X-Tenant-ID: TEN-0001
Content-Type: application/json
```

Request: (varies per schema — this is the STANDARD_FINTECH example)

```
{
  "customer": {
    "full_name": "Rajesh Kumar Sharma",
    "id": "CUST-98271"
  },

```

```
"account": {
  "number": "HDFC-00123456789"
},
"txn": {
  "ref_id": "TXN-2026-061099182",
  "amount": 990000.00,
  "currency": "INR",
  "type": "NEFT_TRANSFER",
  "direction": "DEBIT",
  "timestamp": "2026-06-10T09:30:00+05:30"
},
"counterparty": {
  "account": "ICICI-00987654321",
  "name": "Priya Enterprises",
  "bank": "ICICI Bank"
},
"metadata": {
  "ip": "192.168.1.100",
  "device_id": "MOB-a1b2c3d4e5f6"
},
"risk": {
  "score": 87,
  "reason": "Large outward transfer near reporting threshold"
}
}
```

Response 202 (Accepted — processing begins async):

```
{
  "alert_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "PROCESSING",
  "message": "Alert received and queued for analysis. Check the review queue for results.",
  "estimated_completion_seconds": 8
}
```

Response 401 (Invalid API Key):

```
{
  "error": {
    "code": "INVALID_API_KEY",
    "message": "The provided API key is invalid or has been revoked."
  }
}
```

Response 403 (Tenant Suspended):

```
{
  "error": {
    "code": "TENANT_SUSPENDED",
    "message": "This tenant's access has been suspended. Contact Aegis support."
  }
}
```

Business Logic (runs as FastAPI BackgroundTask after 202 is returned):

1. Mark alert status: **PROCESSING**
2. Look up tenant's active **ingestion_schema**
3. Normalize payload using **field_map** (map tenant keys → Aegis standard fields)
4. Validate required standard fields are present
5. Run PII masker on all **pii_fields** → generate token map → store in **pii_maps**
6. Run 8 AML compliance checks on masked payload → store in **compliance_matches**
7. Build Groq prompt with masked transaction + triggered rules + SAR format instructions
8. Call Groq API → parse response → store draft in **sar_drafts**
9. Mark alert status: **PENDING_REVIEW**
10. Log **ALERT_INGESTED**, **ALERT_MASKING_COMPLETE**, **ALERT_DRAFT_GENERATED** to audit_logs

Domain 5: SAR Review Queue

All endpoints require Bearer JWT with role **TENANT_ADMIN** or **COMPLIANCE_OFFICER**.

GET /api/v1/queue

Query Params: ?status=PENDING_REVIEW&page=1&per_page=20&sort=created_at_desc&search=TXN-2026

Response 200:

```
{
  "items": [
    {
      "id": "uuid",
      "transaction_id": "TXN-2026-061099182",
      "transaction_amount": 990000.00,
      "transaction_currency": "INR",
      "transaction_type": "NEFT_TRANSFER",
      "transaction_timestamp": "2026-06-10T09:30:00Z",
      "risk_score": 87,
      "triggered_rules": ["STRUCTURING", "RISK_SCORE_THRESHOLD"],
      "status": "PENDING_REVIEW",
      "source": "API",
      "created_at": "2026-06-10T09:30:05Z"
    }
  ],
  "total": 12,
```

```
"pending_count": 12
}
```

GET /api/v1/queue/{alert_id}

Full alert detail for the SAR Workspace. Loads all 3 panels of data.

Response 200:

```
{
  "alert": {
    "id": "uuid",
    "transaction_id": "TXN-2026-061099182",
    "status": "PENDING_REVIEW",
    "transaction_amount": 990000.00,
    "transaction_currency": "INR",
    "transaction_type": "NEFT_TRANSFER",
    "transaction_direction": "DEBIT",
    "transaction_timestamp": "2026-06-10T09:30:00Z",
    "risk_score": 87,
    "source": "API",
    "created_at": "2026-06-10T09:30:05Z",
    "masked_payload": {
      "customer_name": "USR_a1b2c3d4",
      "account_id": "ACC_e5f6a7b8",
      "counterparty_account": "ACC_c9d0e1f2",
      "ip_address": "IP_a3b4c5d6",
      ...
    }
  },
  "compliance": {
    "overall_risk": "HIGH",
    "triggered_rules": [
      {
        "rule_id": "STRUCTURING",
        "rule_name": "Structuring / Smurfing Detected",
        "triggered": true,
        "confidence": "HIGH",
        "evidence": {
          "field": "transaction_amount",
          "value": "990000",
          "explanation": "Transaction amount of ₹9,90,000 is just below the ₹10,00,000 reporting threshold. This pattern is a common structuring indicator."
        }
      },
      {
        "rule_id": "RISK_SCORE_THRESHOLD",
        "rule_name": "Risk Score Threshold Exceeded",
        "triggered": true,
        "confidence": "HIGH",
        "evidence": {
```



```
        "field": "risk_score",
        "value": 87,
        "explanation": "Incoming risk score of 87 exceeds the configured
threshold of 75."
    }
},
{
    "rule_id": "RAPID_MOVEMENT",
    "rule_name": "Rapid Fund Movement",
    "triggered": false,
    "confidence": "LOW",
    "evidence": {}
}
],
"sar_draft": {
    "id": "uuid",
    "draft_text": "SUSPICIOUS ACTIVITY REPORT\n\nReporting Entity:
[TENANT_NAME]\nDate of Report: 10 June 2026\n\n1. SUBJECT OF REPORT\nCustomer
Reference: USR_a1b2c3d4\nAccount Reference: ACC_e5f6a7b8\n\n2. NATURE OF
SUSPICIOUS ACTIVITY\nA debit transaction of INR 9,90,000 was flagged on 10 June
2026...",
    "officer_edit_count": 0,
    "last_edited_at": null,
    "llm_model": "llama-3.3-70b-versatile",
    "generation_latency_ms": 2341,
    "created_at": "2026-06-10T09:30:12Z"
}
}
```

PUT /api/v1/queue/{alert_id}/draft

Officer edits the draft text inline.

Request:

```
{
  "draft_text": "SUSPICIOUS ACTIVITY REPORT\n\n[Updated narrative by officer]..."
}
```

Response 200:

```
{
  "message": "Draft updated.",
  "officer_edit_count": 1,
  "last_edited_at": "2026-06-10T10:15:00Z"
}
```

GET /api/v1/queue/{alert_id}/preview-rehydrated

Returns the SAR draft with real PII values restored (for preview only, never stored).

Response 200:

```
{
  "rehydrated_text": "SUSPICIOUS ACTIVITY REPORT\n\nReporting Entity: PayFast India Pvt Ltd\nDate of Report: 10 June 2026\n\n1. SUBJECT OF REPORT\nCustomer Name: Rajesh Kumar Sharma\nAccount Number: HDFC-00123456789\n\n2. NATURE OF SUSPICIOUS ACTIVITY\n...",
  "warning": "This preview contains real PII. Do not share this response. It is for officer review only."
}
```

POST /api/v1/queue/{alert_id}/approve

Officer approves the SAR. Triggers re-hydration, PDF generation, and webhook delivery.

Request: Empty body (officer ID taken from JWT)

Response 200:

```
{
  "message": "SAR approved and delivery initiated.",
  "sar_id": "uuid",
  "approved_at": "2026-06-10T10:20:00Z",
  "delivery": {
    "status": "DELIVERED",
    "destination": "internal_sink",
    "delivery_id": "uuid"
  }
}
```

Background Process (synchronous for MVP — within 5s):

1. Validate alert is in **PENDING_REVIEW** status
2. Retrieve current **draft_text** from **sar_drafts**
3. Retrieve **token_map** from **pii_maps**
4. Re-hydrate: replace all tokens in **draft_text** with real values → store as **rehydrated_text**
5. Generate PDF using reportlab template → store as **pdf_path**
6. Compute HMAC-SHA256 signature over **{sar_id}:{approved_at}:{pdf_base64}**
7. Deliver webhook (to callback URL or internal sink)
8. Update alert status to **APPROVED**
9. Store delivery record in **webhook_deliveries**
10. Log **ALERT_APPROVED**, **SAR_REHYDRATED**, **PDF_GENERATED**, **WEBHOOK_DELIVERED**

POST /api/v1/queue/{alert_id}/reject

Request:

```
{ "reason": "Transaction reviewed – determined to be legitimate payroll  
disbursement." }
```

Response 200:

```
{  
  "message": "Alert rejected and cleared from queue.",  
  "rejection_reason": "Transaction reviewed – determined to be legitimate payroll  
disbursement.",  
  "rejected_at": "2026-06-10T10:20:00Z"  
}
```

Domain 6: Webhook Sink (Built-in Test Receiver)

POST /api/v1/webhooks/sink/{tenant_id_public}

The built-in receiver endpoint. Accepts the same payload the real webhook would receive.

Auth: HMAC verification (X-Aegis-Signature header)

Headers (sent by Aegis WebhookDispatcher):

```
Content-Type: application/json  
X-Aegis-Signature: sha256=a1b2c3d4...  
X-Aegis-Delivery-ID: uuid  
X-Aegis-Timestamp: 1717999200
```

Request Body:

```
{  
  "sar_id": "uuid",  
  "alert_id": "uuid",  
  "approved_at": "2026-06-10T10:20:00Z",  
  "approved_by": "Nikhil Karur",  
  "tenant_id": "TEN-0001",  
  "narrative_text": "SUSPICIOUS ACTIVITY REPORT\n\nReporting Entity: PayFast  
India...",  
  "pdf_base64": "JVBERi0xLjQK...",  
  "compliance_rules_triggered": ["STRUCTURING", "RISK_SCORE_THRESHOLD"],  
}
```

```
"hmac_signature": "sha256=a1b2c3d4..."
}
```

Response 200:

```
{ "status": "received", "event_id": "uuid" }
```

GET /api/v1/webhooks/sink/{tenant_id_public}/events

Returns last 10 events received by the built-in sink for this tenant.

Auth: Bearer JWT (TENANT_ADMIN)

Response 200:

```
{
  "events": [
    {
      "id": "uuid",
      "received_at": "2026-06-10T10:20:03Z",
      "hmac_valid": true,
      "payload": {
        "sar_id": "uuid",
        "approved_at": "2026-06-10T10:20:00Z",
        "narrative_text": "SUSPICIOUS ACTIVITY REPORT...",
        "compliance_rules_triggered": ["STRUCTURING", "RISK_SCORE_THRESHOLD"]
      }
    }
  ]
}
```

Domain 7: Simulator

POST /api/v1/simulator/submit-test-alert

Generates a realistic synthetic alert and injects it through the full pipeline.

Auth: Bearer JWT (TENANT_ADMIN or COMPLIANCE_OFFICER)

Request:

```
{
  "scenario": "STRUCTURING",
  "custom_risk_score": 87
}
```

Valid scenarios: **STRUCTURING**, **RAPID_MOVEMENT**, **HIGH_RISK_TYPE**, **VELOCITY**, **DEFAULT**

Response 202:

```
{
  "alert_id": "uuid",
  "message": "Test alert injected. Check the review queue in ~8 seconds.",
  "scenario_used": "STRUCTURING",
  "synthetic_transaction_id": "TEST-TXN-2026061099999"
}
```

Business Logic:

1. Generate a synthetic transaction payload matching the tenant's active schema template
2. For the requested scenario, populate transaction fields to trigger the corresponding AML rule
3. Inject directly into the pipeline (same as if it came from **POST /alerts/ingest**)
4. Flag alert with **source = SIMULATOR**

Webhook Payload Verification (Client-Side Reference)

Clients verifying the HMAC on received webhooks:

```
import hmac
import hashlib

def verify_aegis_webhook(payload_bytes: bytes, signature_header: str, secret: str)
-> bool:
    expected = hmac.new(
        secret.encode('utf-8'),
        payload_bytes,
        hashlib.sha256
    ).hexdigest()
    received = signature_header.replace("sha256=", "")
    return hmac.compare_digest(expected, received)
```

The **X-Aegis-Signature** header format: **sha256=<hex_digest>** The HMAC is computed over the **raw request body bytes** (not parsed JSON).

Rate Limits (MVP defaults, not enforced per-tenant)

Endpoint	Limit
POST /auth/login	10/min per IP
POST /alerts/ingest	60/min per tenant
POST /simulator/submit-test-alert	10/min per tenant
All other endpoints	120/min per user